

Video Dialogue Localization System — Design & Approach

An end-to-end pipeline that takes a **public video URL** and a **target line of dialogue**, and returns the exact frame where that line is spoken — timestamp, frame number, matched text, confidence score, and the frame image — with no human ever inspecting the video.

Reference target: ok.ru/video/248244667877 — *The Adventures of Sherlock Holmes: A Scandal in Bohemia* (54 min) — locating "My mind rebels at stagnation."

Result: 00:05:24.970 · frame 7795 · 100% confidence · 2.6 min cold · 0.22s warm

Repo: github.com/abinayaa0/Quest1 · **Stack:** Python · yt-dlp · FFmpeg · Faster-Whisper (CTranslate2) · RapidFuzz · Streamlit

Phase 1 — Problem statement analysis

Before any code, the brief was decomposed into what is actually specified, what is ambiguous, and what has to be inferred.

1.1 Input / output contract

Input

1. **Video URL** — any publicly accessible video link, not just ok.ru. The provided example has no subtitles or captions.
2. **Target dialogue** — the line to locate, at its **first** occurrence.

Output

1. **Timestamp** of the identified frame, formatted [HH:MM:SS.sss](#)
2. **Frame number**, where applicable
3. **Extracted dialogue text**
4. **The corresponding video frame** as an image

1.2 Functional requirements

ID	Requirement
FR1	Accept a publicly accessible video URL
FR2	Retrieve and process the video without the evaluator manually inspecting it
FR3	Locate the specified dialogue within the video
FR4	Identify the first occurrence
FR5	Provide the corresponding timestamp
FR6	Provide the frame number, where applicable
FR7	Provide the text associated with the identified dialogue
FR8	Return the actual video frame as an image

Localization requirement. The system must determine *where in the video* to look automatically — this is the requirement that rules out any design depending on a human narrowing the search first.

1.3 Robustness requirements

The brief explicitly asks for reasonable robustness to:

Variation	How it is handled
Video quality (high / medium / low)	ASR operates on the audio track, which degrades far more gracefully than visual text
Resolution (720p, 1080p, ...)	Ingestion caps at $\leq 720p$; frame extraction probes actual resolution rather than assuming
Frame rate	Rational FPS strings preserved exactly; VFR detected explicitly; PTS-based seeking (Phase 6)
Appearance of the dialogue	See interpretation below

1.4 Interpreting "appearance of the dialogue"

The phrase is ambiguous, potentially referring to on-screen subtitles or spoken dialogue. However, inspecting the video reveals no subtitles, proving the task requires spoken-dialogue temporal localization. Because an audio-based approach works universally even when subtitles are absent, this interpretation is both practically necessary and more general.

1.5 Ambiguities vs engineering considerations

Separating what the brief leaves open from what the implementation must simply handle:

Left unspecified by the brief	Practical cases the implementation handles
Supported video platforms	Missing audio track
Maximum video duration	Redirects
Direct media URLs vs webpage URLs	Download failures
Multiple audio-track behaviour	Network failures / rate limiting
	Varying media formats, resolutions, frame rates

1.6 Other requirements

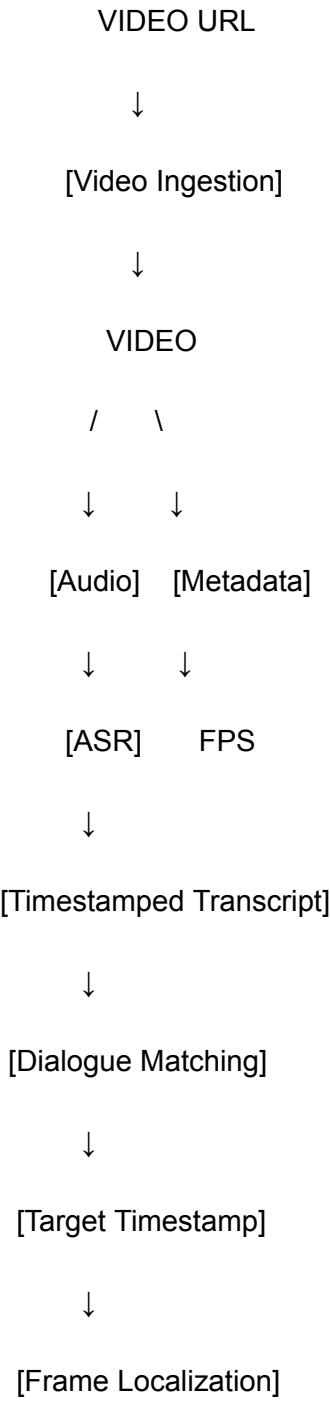
- **Generalization** — any dialogue text, any video URL. The evaluator states they may substitute a different video and line, so nothing may be tuned to the Sherlock file.
- **Uncertainty handling** — the system must express confidence rather than silently returning a wrong answer.

1.7 Non-functional requirements

Requirement	Source
Reasonable processing time	Derived
Reproducibility	Derived
Maintainable, modular code	Derived

Requirement	Source
Unit tests	Explicit (evaluator conversation)
Deployment	Optional / encouraged, not required

1.8 Baseline architecture derived from the above





[Frame Image]

Everything below is the elaboration of this skeleton, plus the V2 optimization layer that sits inside the ASR stage.

1. Core idea

A 54-minute video contains ~3,300 seconds of speech. The target query is ~3 seconds long. Running an accurate, word-timestamped ASR model across the whole file to answer that question wastes 99.9% of the compute.

The final pipeline (V2) therefore splits the work in two:

- a **cheap pass** over all the audio to find *roughly where to look*, and
- an **accurate pass** over only the ~17-second candidate windows to find *exactly where it is*.

That split wasn't the starting point — V1 was a single full-accuracy pass, and the sections below document what was tried at each phase and why it changed.

2. Module architecture at a glance

#	Phase	Module	Output artifact
1	Video ingestion	<code>src/ingestion/</code>	<code>output/<id>.mp4</code> + <code>VideoMetadata</code>
2	Audio extraction	<code>src/audio/</code>	<code>output/<id>.wav</code> (16 kHz mono PCM) + <code>AudioMetadata</code>
3	Coarse ASR	<code>src/asr/</code>	<code>output/<id>_transcript_coarse_base.json</code> (segments)

#	Phase	Module	Output artifact
4	Candidate retrieval	<code>src/matching/</code>	Top-K candidate windows (<code>start</code> , <code>end</code> , <code>score</code>)
5	Fine ASR	<code>src/asr/</code>	<code>output/<id>_v2_fine_cache.json</code> (word timestamps)
6	Dialogue matching	<code>src/matching/</code>	<code>MatchResult</code> (timestamps, text, confidence, tier)
7	Frame extraction	<code>src/frame_extraction/</code>	<code>output/frames/frame_324_97.jpg</code> + <code>FrameResult</code>
—	Orchestration & logging	<code>src/pipeline.py</code> , <code>app.py</code> , <code>cli.py</code> , <code>cli_v2.py</code>	<code>localization_result.json</code> , <code>query_history.xlsx/.csv</code>

Each module exposes a single public entry point (`ingest_video()`, `extract_audio()`, `transcribe_audio()`, `match_dialogue()`, `extract_frame()`), has its own typed error hierarchy, and is unit-testable in isolation.

3. Phase-by-phase design

Phase 1 — Video ingestion (`src/ingestion/`)

Responsibility: resolve an unverified public URL into a decodable local `.mp4`, validate it, and capture frame-accurate timing metadata.

What was tried first. Direct HTTP download via `urllib` / plain `yt-dlp` defaults. Against the assignment's OK.ru target this failed immediately:

Output: `output/248244667877.mp4` — 1.04 GB, 960×720, 24000/1001 fps, h264/aac, 3,261.8s. Downloaded in 133.5s.

Verified across OK.ru, YouTube (AV1/Opus), and direct MP4;

Phase 2 — Audio extraction (`src/audio/`)

Responsibility: convert the video into the exact audio format ASR engines expect.

What was tried first. Letting Whisper ingest the `.mp4` directly and resample internally.

Why it was changed. In-engine resampling adds per-run overhead on every retry and every model comparison, and gives no cached intermediate. Extracting once to a canonical WAV makes every downstream experiment cheaper and reproducible.

Final design: a single FFmpeg pass producing 16 kHz, 1-channel, uncompressed 16-bit PCM — mono because ASR models operate on 1-D waveforms.

Output: `output/248244667877.wav` — 99.54 MB, 3,261.8s, extracted in **7.87s**.

Phase 3 — ASR / speech recognition (`src/asr/`)

Responsibility: turn 54 minutes of speech into searchable, timestamped text.

What was tried first — engine. OpenAI's reference `whisper` package on CPU. Too slow to iterate on.

Why it was changed. Switched to **Faster-Whisper** (CTranslate2 backend) with `compute_type="int8"`, which quantizes to 8-bit integer inference and is dramatically faster on CPU with no GPU requirement. Silero VAD filtering (`vad_filter=True`) was enabled to skip silence rather than transcribe it.

What was tried next — model size. All four sizes were benchmarked on the same 54-minute Sherlock audio to find where accuracy stopped improving relative to cost:

HOW WE CHOSE THE FASTER WHISPER MODEL? Logical comparisons:
Here is the model comparison formatted cleanly into a readable table:

Model	Type	Runs CPU ?	Open Source	Word Timestamps	Accuracy	Pros	Cons
Faster-Whisper	Whisper optimized inference	✓ Yes	✓ Yes	✓ Yes	★★★★	Fast, accurate, easy integration, CPU-friendly	Large models can still be slow on CPU
Whisper (original)	OpenAI Whisper	✓ Yes	✓ Yes	⚠ Requires options	★★★★	Very proven, strong multilingual support	Slower inference
Vosk	Kaldi-based ASR	✓ Yes	✓ Yes	✓ Yes	★★★★	Lightweight, fast CPU inference	Lower accuracy on noisy/movie audio
wav2vec2	Self-supervised ASR	✓ Yes	✓ Yes	⚠ Requires alignment	★★★★★	Strong speech representation	Timestamp pipeline is more complex

NVIDIA Parakeet	NeMo ASR model	⚠ Possible	✓ Yes	✓ Yes	★★★★	Very strong accuracy and speed	More GPU-oriented ecosystem
AssemblyAI	Cloud ASR API	N/A	✗ API	✓ Yes	★★★★	Easy integration, excellent timestamps	Paid dependency

Model	Full-audio transcription time	Word timestamps	Verdict
<code>tiny</code>	fastest	yes	Too many word substitutions; degraded fuzzy match scores
<code>base</code>	123.1s	disabled for coarse pass	Chosen for coarse search — segment text accurate enough to <i>locate</i> the line
<code>small</code>	419.18s	yes	Chosen for fine pass — accurate word boundaries
<code>medium</code>	prohibitive on CPU	yes	No measurable accuracy gain over <code>small</code> for this task

That table is the origin of the whole V2 design: `base` is ~3.4× faster than `small` on the same audio and is *good enough to find the neighbourhood*, while `small` is only needed to pin down word boundaries inside it.

What was tried for long files. Sequential single-stream transcription of 50+ minute audio hit two walls — CTranslate2 decodes single-threaded per stream, and NumPy STFT buffers spiked into `ArrayMemoryError` on long files.

Why it was changed. Audio is now sliced into 600-second chunks before transcription, with word timestamps offset back onto the global timeline (`chunk_start_time + seg.start`). Multi-core execution uses a **single shared** `WhisperModel` across a `ThreadPoolExecutor`. **Increased upto 8 cores.**

Parallelization benchmark (33.6-min YouTube video):

Mode	Time	RAM
Sequential	624.55s (10.4 min)	~461 MB
Parallel, 2 workers	563.06s (9.3 min)	~461 MB (shared weight matrix)

Output: `output/248244667877_transcript_coarse_base.json` — 623 segments, language `en` (p=0.96), 123.1s.

Phase 4 — Candidate retrieval (`src/matching/`) · *V2 only*

Responsibility: narrow 3,261 seconds down to the few regions worth transcribing accurately.

What was tried first. Nothing — V1 had no retrieval stage. It ran `small` with word timestamps across the entire file and searched the result.

Why it was changed. That spends 419 seconds of CPU to answer a question about 3 seconds of audio.

Final design: RapidFuzz sequence matching over the *coarse* segment transcript, returning the **top-K = 3** highest-scoring regions, each padded by **±5.0s** so a line straddling a segment boundary isn't clipped. Only those windows proceed.

Output: 3 candidate windows (~17s each) — roughly **1% of the original audio** forwarded to the expensive model.

Phase 5 — Fine ASR (`src/asr/`) · V2 only

Responsibility: produce word-level timestamps, but only where they're needed.

Final design: FFmpeg extracts each candidate window as a WAV sub-region; `small` transcribes it with `word_timestamps=True`; resulting word timings are re-offset onto the global timeline (`word.start + pad_start`).

Accuracy cost of the shortcut: `00:05:25.110` (V1, full-file `small`) vs `00:05:24.970` (V2, windowed `small`) — a **0.14s / 4-frame delta**, both at 100% confidence. The 62% speedup is effectively free.

Output: `output/248244667877_v2_fine_cache.json` — word timestamps for candidate regions only.

Phase 6 — Dialogue matching (`src/matching/`)

Responsibility: map the query text to precise `start_time` / `end_time`.

What was tried first — segment-level matching. Score the query against each Whisper segment, take the best.

Why it was changed. Whisper's segment boundaries are silence-driven, not sentence-driven, so a target line routinely straddles two segments ("`My mind rebels`" / "`at stagnation`"). Segment-only matching fails exactly on the cases that matter.

What was tried next — semantic embeddings. Considered and deliberately rejected. The task is **exact dialogue retrieval** — where was *this specific line* spoken — not semantic search. An embedding model ranks "*I hate sitting still*" as a strong match for "*My mind rebels at stagnation*" because the concepts are adjacent. Lexical fuzzy matching won't, because the words aren't. It also avoids PyTorch weights, a vector store, and non-deterministic scoring.

Final design:

- Normalization: lowercasing, punctuation removal, whitespace collapsing
- **Word-level sliding window** of length $L \pm 2$ across the word stream, not segments
- `fuzz.ratio` over `fuzz.token_set_ratio` — ratio preserves word order; order-insensitive scoring reintroduces the same false positives from the opposite direction
- **First-occurrence rule** enforced — the question is *when was this first said*, not *where does it best match*

- **Boundary refinement** — trim the matched window back to the target's true first and last words:

Window ["in", "time.", "My", "mind", "rebels", "at", "stagnation."]

Trimmed ["My", "mind", "rebels", "at", "stagnation."]

→ start_time = word["My"].start = 325.11s

→ confidence 87.5% → 100.0%

Handling ambiguity — every match carries a confidence tier:

Score	Tier
> 90%	Strong match
80–90%	Acceptable
70–80%	Needs review
< 70%	Reject

These boundaries are empirically derived, not arbitrary — see §5.

Output: `MatchResult(start_time=324.97, matched_text="My mind rebels at stagnation", confidence=100.0, tier="strong")`

Phase 7 — Frame extraction ([src/frame_extraction/](#))

Responsibility: turn a timestamp into the actual image.

What was tried first. `frame_number = timestamp × fps`, then seek to that index.

Why it was changed. Two independent failures:

1. **Rational frame rates.** The target runs at $24000/1001 \approx 23.976$ fps. Multiplying a float FPS by a 3,200-second timestamp accumulates rounding drift measured in whole frames by the end of the file.
2. **Variable frame rate.** HLS streams, web downloads and screen recordings fluctuate frame intervals dynamically; a constant average FPS drifts by seconds across a

50-minute video. `ffprobe` detects this explicitly (`is_vfr`) by comparing `r_frame_rate` against `avg_frame_rate`.

Final design: the timestamp is the source of truth. FFmpeg seeks by embedded **Presentation Timestamp** (`ffmpeg -ss <t> -i <video> -frames:v 1`), which is codec- and VFR-agnostic and works across MP4, MKV, WebM, H.264, H.265, VP9 and AV1 without custom decoders. A **20 ms pre-roll offset** (`seek_ts = max(0, t - 0.02)`) compensates for FFmpeg advancing to the *next* frame when the target lands a millisecond past a frame's PTS.

Where a frame index is reported, it uses integer math on the preserved rational fraction:

$$\text{Frame} = \left\lfloor t \times \frac{\text{num}}{\text{den}} \right\rfloor$$

If it can't be derived without a full index scan, the system returns `frame_number = None` rather than guessing.

Output: `output/frames/frame_324_97.jpg` — 960×720, probed and verified.

Caching layer

Two independent disk caches make repeat queries nearly free:

Layer	File	Contents
Coarse	<code><id>_transcript_coarse_base.json</code>	Full-video segment transcript
Fine	<code><id>_v2_fine_cache.json</code>	Candidate-window word timestamps

Warm query flow: load coarse transcript → retrieve candidates → run fine ASR only if that window isn't cached → match → return.

Repeat query	Time	RAM
Without caching	161.22s	~461 MB
With dual-layer cache	0.216s	~15 MB

4. Pipeline evolution & consolidated benchmarks

V1 — ingest → extract audio → full-file **small** ASR with word timestamps → sliding-window match → frame extract. **V2** — ingest → extract audio → full-file **base** coarse ASR → top-K retrieval → windowed **small** fine ASR → match → frame extract, with both stages cached.

Video	Mode	ASR config	Runtime	Match	Confidence	Timestamp	Frame	RAM
Sherlock, 54 min	V1 cold	small , full audio	419.18s (7.0 m)	✓	100%	00:05:25.110	frame_325_11.jpg	~461 MB
Sherlock, 54 min	V2 cold	base → small	159.08s (2.6 m)	✓	100%	00:05:24.970	frame_324_97.jpg	~461 MB
Sherlock, 54 min	V2 warm	disk cache	0.216s	✓	100%	00:05:24.970	frame_324_97.jpg	~15 MB
YouTube, 33.6 min	V1 sequential	small , full audio	624.55s (10.4 m)	✓	100%	00:09:26.570	frame_566_57.jpg	~461 MB
YouTube, 33.6 min	Parallel	small , 2 workers	563.06s (9.3 m)	✓	100%	00:09:26.550	frame_566_55.jpg	~461 MB

Headline results: 62% faster cold runs · 99.8% faster warm runs · 100% match confidence preserved · 0.14s accuracy delta.

Additional verified retrievals: "To Sherlock Holmes she was always the woman" → 00:01:58.490, frame 2841, 88.1% · "Imagine you ran a store" (YouTube) → 00:09:26.570, frame 16980, 100%.

benchmark_v1_vs_v2.py reruns this whole matrix side-by-side and prints it, so the numbers are reproducible rather than hand-recorded.

5. Robustness testing

Latency benchmarks say nothing about whether retrieval is *correct*. A separate pass tested speech domains and query classes to find where it breaks.

Domains: TV/movie dialogue (*Friends*, *Sherlock Holmes*), gaming commentary (fast, slang-heavy), podcasts (long-form, multi-speaker), accented English, formal single-speaker talk.

Query class	Example	Expected	Result
Exact	having my worst fear realized	TRUE	✓ 100%, 258.75s
Case variant	Having My Worst Fear Realized	TRUE	✓ same occurrence
Partial	worst fear realized	TRUE	✓ 259.19s
Negative	the spaceship landed on mars	FALSE	✓ rejected
Cross-video negative	the blue carbuncle (wrong video)	FALSE	✓ rejected

All positive variants converged on the same occurrence, confirming normalization and partial-query handling.

The finding that mattered. Hard-negative testing exposed a false-positive class pure latency benchmarking would never surface:

Query	ASR text	Score
opposite of jam is man	opposite of man?	81.1%
im in a lot of pain here my face is hurt	I'm in a lot of pain here, all right?	81.1%

Both share a matching prefix but assert something different. The matcher was answering "*does this look similar?*" rather than "*did the speaker say this?*", returning TRUE at ~80% on long queries with partial lexical overlap.

This cluster is what produced the 80/90 tier boundaries and the word-boundary trimming in Phase 6 — it's why 80–90% reads *acceptable* rather than *match*, and why anything under 70% is rejected outright.

6. Interfaces

Streamlit dashboard (`app.py`) — paste a URL or upload a file, enter the dialogue query, watch processing status, then view the matched timestamp, matched text and confidence score alongside the extracted frame, with a download button for the image. A sidebar keeps query history for the session, and the full log exports as Excel or CSV.

CLIs — `cli.py` offers interactive mode selection ([1] Standard V1, [2] Coarse-to-Fine V2) or flags (`--url`, `--query`, `--mode`); `cli_v2.py` is a V2-only shortcut; `benchmark_v1_vs_v2.py` runs the comparison matrix.

Automatic history logging — every query, from any interface, writes 14 execution metrics to `query_history.xlsx` / `.csv`: total time, per-stage ASR times, model used, candidate info, match score, frame info, video info, and a cold/warm flag. This is what made §4 reproducible instead of hand-tallied.

7. Testing

67 unit and integration tests pass in 0.51s across `test_unit.py`, `test_audio_unit.py`, `test_asr_unit.py`, `test_matching_unit.py`, `test_frame_unit.py` and `test_coarse_to_fine_unit.py` — covering every module plus mocked failure modes (invalid URLs, missing audio/video streams, 404 video IDs, `ffprobe` failures, download timeouts), and real-media integration runs against live Faster-Whisper inference.

8. Limitations

- **Cold-run latency** (~2.6 min for 54 minutes) is still dominated by the full-file coarse pass. Caching only helps repeats.
 - **ASR errors propagate** — a mis-transcribed word is a mis-searched word.
-

LLM and coding-assistant prompts used during development are documented separately in [PROMPTS.md](#).